网络流封装

update: 2026.4.19 更新了 HopcroftKarp 算法模板

Dinic 最大流

```
template<typename T> struct Flow_ {
    const int n;
    const T inf = numeric_limits<T>::max();
    struct Edge {
        int to;
        T w;
        Edge(int to, T w) : to(to), w(w) {}
    };
    vector<Edge> ver;
    vector<vector<int>> h;
    vector<int> cur, d;

    Flow_(int n) : n(n), h(n + 1) {}
    void add(int u, int v, T c)
    {
        h[u].push_back(ver.size());
        ver.emplace_back(v, c);
        h[v].push_back(ver.size());
        ver.emplace_back(u, 0);
    }
    bool bfs(int s, int t)
    {
        d.assign(n + 1, -1);
        d[s] = 0;
        queue<int> q;
        q.push(s);
        while(!q.empty())
        {
            auto x = q.front();
            q.pop();
            for(auto it: h[x])
            {
                auto [y, w] = ver[it];
                if(w && d[y] == -1)
                {
                    d[y] = d[x] + 1;
                    if(y == t) return true;
                    q.push(y);
                }
            }
```

```
        }
        return false;
    }
    T dfs(int u, int t, T f)
    {
        if(u == t) return f;
        auto r = f;
        for(int &i = cur[u]; i < h[u].size(); i++)
        {
            auto j = h[u][i];
            auto &[v, c] = ver[j];
            auto &[u, rc] = ver[j ^ 1];
            if(c && d[v] == d[u] + 1)
            {
                auto a = dfs(v, t, std::min(r, c));
                c -= a;
                rc += a;
                r -= a;
                if(!r) return f;
            }
        }
        return f - r;
    }
    T work(int s, int t)
    {
        T ans = 0;
        while(bfs(s, t))
        {
            cur.assign(n + 1, 0);
            ans += dfs(s, t, inf);
        }
        return ans;
    }
};
using Flow = Flow_<int>;
```

ISAP 最大流

```
template<typename T> struct Flow_ {
    const int n;
    const T inf = numeric_limits<T>::max();
    struct Edge {
        int to;
        T cap, flow;
        Edge(int _to = 0, T _cap = 0) :
            to(_to), cap(_cap), flow(0) {}
    };
    vector<Edge> ver;
    vector<vector<int>> h;
    vector<int> dep, gap, cur;
    vector<int> stk, que;

    Flow_(int n) : n(n), h(n + 1) {}
    void addedge(int u, int v, int w)
    {
        h[u].push_back(ver.size());
        ver.emplace_back(v, w);
        h[v].push_back(ver.size());
        ver.emplace_back(u, 0);
    }
    void bfs(int s, int t)
    {
        dep.assign(n + 1, -1);
        gap.assign(n + 2, 0);
        que.assign(1, 0);
        gap[0] = 1;
        int hh = 0, tt = 0;
        dep[t] = 0, que[0] = t;
        while(hh <= tt)
        {
            int u = que[hh ++];
            for(int i = 0; i < h[u].size(); i++)
            {
                int j = h[u][i];
                auto &[v, c, f] = ver[j];
                if(dep[v] != -1) continue;
                if((int)que.size() == tt + 1) que.push_back(0);
                que[++ tt] = v;
                dep[v] = dep[u] + 1;
                gap[dep[v]] ++;
            }
        }
```

```
    }
    T work(int s, int t)
    {
        bfs(s, t);
        cur.assign(n + 1, 0);
        int u = s, top = 0;
        T ans = 0;
        while(dep[s] < n)
        {
            if(u == t)
            {
                T Min = inf;
                int inser;
                for(int i = 0; i < top; i++)
                    if(Min > ver[stk[i]].cap - ver[stk[i]].flow)
                    {
                        Min = ver[stk[i]].cap - ver[stk[i]].flow;
                        inser = i;
                    }
                for(int i = 0; i < top; i++)
                {
                    ver[stk[i]].flow += Min;
                    ver[stk[i] ^ 1].flow -= Min;
                }
                ans += Min;
                top = inser;
                u = ver[stk[top] ^ 1].to;
                continue;
            }
            bool flag = false;
            int _v;
            for(int &i = cur[u]; i < h[u].size(); i++)
            {
                auto j = h[u][i];
                auto &[v, c, f] = ver[j];
                if(c - f && dep[v] + 1 == dep[u])
                {
                    flag = true;
                    _v = v;
                    break;
                }
            }
            if(flag)
            {
                if(stk.size() == top) stk.push_back(0);
                stk[top ++] = h[u][cur[u]];
                u = _v;
```

```
                continue;
            }
            int Min = n;
            for(int i = 0; i < h[u].size(); i++)
            {
                auto j = h[u][i];
                auto &[v, c, f] = ver[j];
                if(c - f && dep[v] < Min)
                {
                    Min = dep[v];
                    cur[u] = i;
                }
            }
            gap[dep[u]] --;
            if(!gap[dep[u]]) return ans;
            dep[u] = Min + 1;
            gap[dep[u]] ++;
            if(u != s) u = ver[stk[-- top] ^ 1].to;
        }
        return ans;
    }
};
using Flow = Flow_<int>;
```

HLPP 预流推进

```
template<typename T> struct PushRelabel {
    const int inf = 0x3f3f3f3f;
    const T INF = 0x3f3f3f3f3f3f3f3f;
    struct Edge {
        int to, cap, flow, anti;
        Edge(int v = 0, int w = 0, int id = 0) :
            to(v), cap(w), flow(0), anti(id) {}
    };
    vector<vector<Edge>> e;
    vector<vector<int>> gap;
    vector<T> ex;
    vector<bool> ingap;
    vector<int> h;
    int n, gobalcnt, maxH = 0;
    T maxflow = 0;

    PushRelabel(int n) : n(n), e(n + 1), ex(n + 1), gap(n + 1) {}
    void addedge(int u, int v, int w)
    {
        e[u].push_back({v, w, (int)e[v].size()});
        e[v].push_back({u, 0, (int)e[u].size() - 1});
    }
    void PushEdge(int u, Edge &edge)
    {
        int v = edge.to, d = min(ex[u], 1ll * edge.cap - edge.flow);
        ex[u] -= d;
        ex[v] += d;
        edge.flow += d;
        e[v][edge.anti].flow -= d;
        if(h[v] != inf && d > 0 && ex[v] == d && !ingap[v])
        {
            ++ gobalcnt;
            gap[h[v]].push_back(v);
            ingap[v] = 1;
        }
    }
    void PushPoint(int u)
    {
        for(auto k = e[u].begin(); k != e[u].end(); k++)
        {
            if(h[k->to] + 1 == h[u] && k->cap > k->flow)
            {
                PushEdge(u, *k);
                if(!ex[u]) break;
```

```
            }
        }
        if(!ex[u]) return;
        if(gap[h[u]].empty())
        {
            for(int i = h[u] + 1; i <= min(maxH, n); i++)
            {
                for(auto v: gap[i])
                    ingap[v] = 0;
                gap[i].clear();
            }
        }
        h[u] = inf;
        for(auto [to, cap, flow, anti]: e[u])
        {
            if(cap > flow)
                h[u] = min(h[u], h[to] + 1);
        }
        if(h[u] >= n) return;
        maxH = max(maxH, h[u]);
        if(!ingap[u])
        {
            gap[h[u]].push_back(u);
            ingap[u] = 1;
        }
    }
    void init(int t, bool f = 1)
    {
        ingap.assign(n + 1, 0);
        for(int i = 1; i <= maxH; i++)
            gap[i].clear();
        gobalcnt = 0, maxH = 0;
        queue<int> q;
        h.assign(n + 1, inf);
        h[t] = 0, q.push(t);
        while(q.size())
        {
            int u = q.front();
            q.pop(), maxH = h[u];
            for(auto &[v, cap, flow, anti]: e[u])
            {
                if(h[v] == inf && e[v][anti].cap > e[v][anti].flow)
                {
                    h[v] = h[u] + 1;
                    q.push(v);
                    if(f)
                    {
```

```
                        gap[h[v]].push_back(v);
                        ingap[v] = 1;
                    }
                }
            }
        }
    }
    T work(int s, int t)
    {
        init(t, 0);
        if(h[s] == inf) return maxflow;
        h[s] = n;
        ex[s] = INF;
        ex[t] = -INF;
        for(auto k = e[s].begin(); k != e[s].end(); k++)
            PushEdge(s, *k);
        while(maxH > 0)
        {
            if(gap[maxH].empty())
            {
                maxH --;
                continue;
            }
            int u = gap[maxH].back();
            gap[maxH].pop_back();
            ingap[u] = 0;
            PushPoint(u);
            if(gobalcnt >= 10 * n)
                init(t);
        }
        ex[s] -= INF;
        ex[t] += INF;
        return maxflow = ex[t];
    }
};
```

EK 费用流

```
struct MinCostFlow {
    using LL = long long;
    using PII = pair<LL,int>;
    const LL INF = numeric_limits<LL>::max();
    struct Edge {
        int v, c, f;
        Edge(int v, int c, int f) : v(v), c(c), f(f) {}
    };
    const int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<LL> h, dis;
    vector<int> pre;

    MinCostFlow(int n) : n(n), g(n + 1) {}
    void add(int u, int v, int c, int f) // c 流量, f 费用
    {
        g[u].push_back(e.size());
        e.emplace_back(v, c, f);
        g[v].push_back(e.size());
        e.emplace_back(u, 0, -f);
    }
    bool dijkstra(int s, int t)
    {
        dis.assign(n + 1, INF);
        pre.assign(n + 1, -1);
        priority_queue<PII, vector<PII>, greater<PII>> que;
        dis[s] = 0;
        que.emplace(0, s);
        while(!que.empty())
        {
            auto [d, u] = que.top();
            que.pop();
            if(dis[u] < d) continue;
            for(int i: g[u])
            {
                auto [v, c, f] = e[i];
                if(c > 0 && dis[v] > d + h[u] - h[v] + f)
                {
                    dis[v] = d + h[u] - h[v] + f;
                    pre[v] = i;
                    que.emplace(dis[v], v);
                }
            }
```

```
        }
        return dis[t] != INF;
    }
    pair<int,LL> flow(int s, int t)
    {
        int flow = 0;
        LL cost = 0;
        h.assign(n + 1, 0);
        while(dijkstra(s, t))
        {
            for(int i = 1; i <= n; i++)
                h[i] += dis[i];
            int aug = numeric_limits<int>::max();
            for(int i = t; i != s; i = e[pre[i] ^ 1].v)
                aug = min(aug, e[pre[i]].c);
            for(int i = t; i != s; i = e[pre[i] ^ 1].v)
            {
                e[pre[i]].c -= aug;
                e[pre[i] ^ 1].c += aug;
            }
            flow += aug;
            cost += LL(aug) * h[t];
        }
        return {flow, cost};
    }
};
```

HopcroftKarp

HK算法专门用于求解二分图最大匹配，理论复杂度 $O(E\sqrt{V})$ 。Dinic处理一般网络流时间复杂度为 $O(E \cdot V^2)$ ，处理二分图匹配时，理论复杂度为 $O(E\sqrt{V})$ ，但实际运行中常数为 HK 算法大

```
template<typename T> struct HopcroftKarp_ {
    vector<vector<int>> g;
    vector<int> pa, pb;
    vector<T> dist;
    int n, m;
    const T inf = numeric_limits<T>::max();

    // 构造函数：传入左部点数量 n，右部点数量 m
    HopcroftKarp_(int n, int m) : n(n), m(m) {
        g.resize(n + 1);
        pa.assign(n + 1, -1);
        pb.assign(m + 1, -1);
        dist.resize(n + 1);
    }

    void add(int x, int y) {
        g[x].push_back(y);
    }

    bool bfs() {
        queue<int> q;
        for (int u = 1; u <= n; ++u) {
            if (pa[u] == -1) {
                dist[u] = 0;
                q.push(u);
            } else {
                dist[u] = inf;
            }
        }
        bool found = false;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) {
                if (pb[v] == -1) {
                    found = true;
                } else if (dist[pb[v]] == inf) {
                    dist[pb[v]] = dist[u] + 1;
                    q.push(pb[v]);
                }
            }
```

```
        }
        return found;
    }

    bool dfs(int u) {
        for (int v : g[u]) {
            if (pb[v] == -1 || (dist[pb[v]] == dist[u] + 1 && dfs(pb[v]))) {
                pa[u] = v;
                pb[v] = u;
                return true;
            }
        }
        dist[u] = inf;
        return false;
    }

    T work() {
        T res = 0;
        while (bfs()) {
            for (int i = 1; i <= n; ++i) {
                if (pa[i] == -1) {
                    res += dfs(i);
                }
            }
        }
        return res;
    }
};

using HopcroftKarp = HopcroftKarp_<int>;
```